



Topic : PCI Bus, SCSI Bus, USB, Data transfer techniques DMA & I/O ^{process}

Objective : To get the Knowledge about Interfaces, Data transfer, DMA

Outcomes : Demonstrate different type of communication with I/O devices and Standard I/O interfaces

Q1 Explain I/O program controlled Transfer vs DMA Transfer in detail. write the names of flags found in the 8085 microprocessors?

Ans - I/O program controlled transfer.

The I/O program controlled transfer also known as programmed I/O (PIO) data transfer between the CPU and I/O devices is managed directly by the CPU. The CPU executes program instructions to transfer data between memory and I/O devices, controlling each step of the transfer process.

Steps : • The CPU initiates the I/O transfer by issuing specific instructions to transfer data between memory and I/O ports.

- Each transfer operations involves multiple CPU instructions, to transfer data including loading data from memory into registers, performing data manipulation or processing, and writing data to or reading data from I/O ports.
- The CPU polls or checks the status of the I/O devices to ensure that the data transfer is completed successfully before proceeding with the next instructions.

02

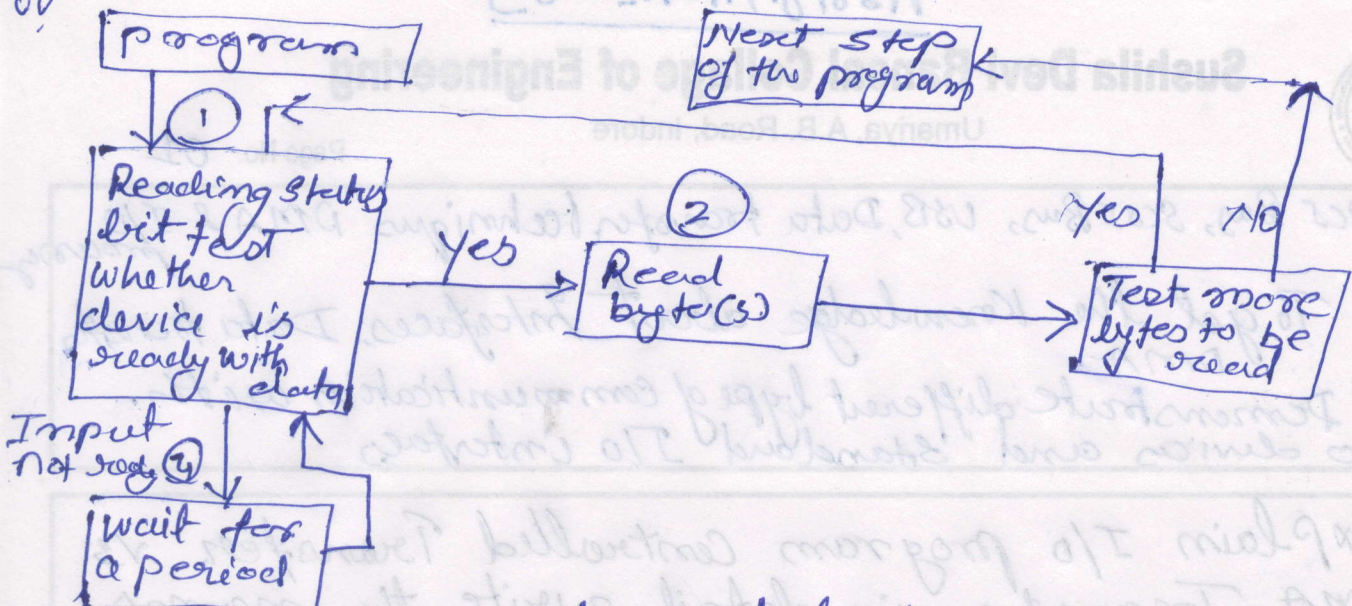
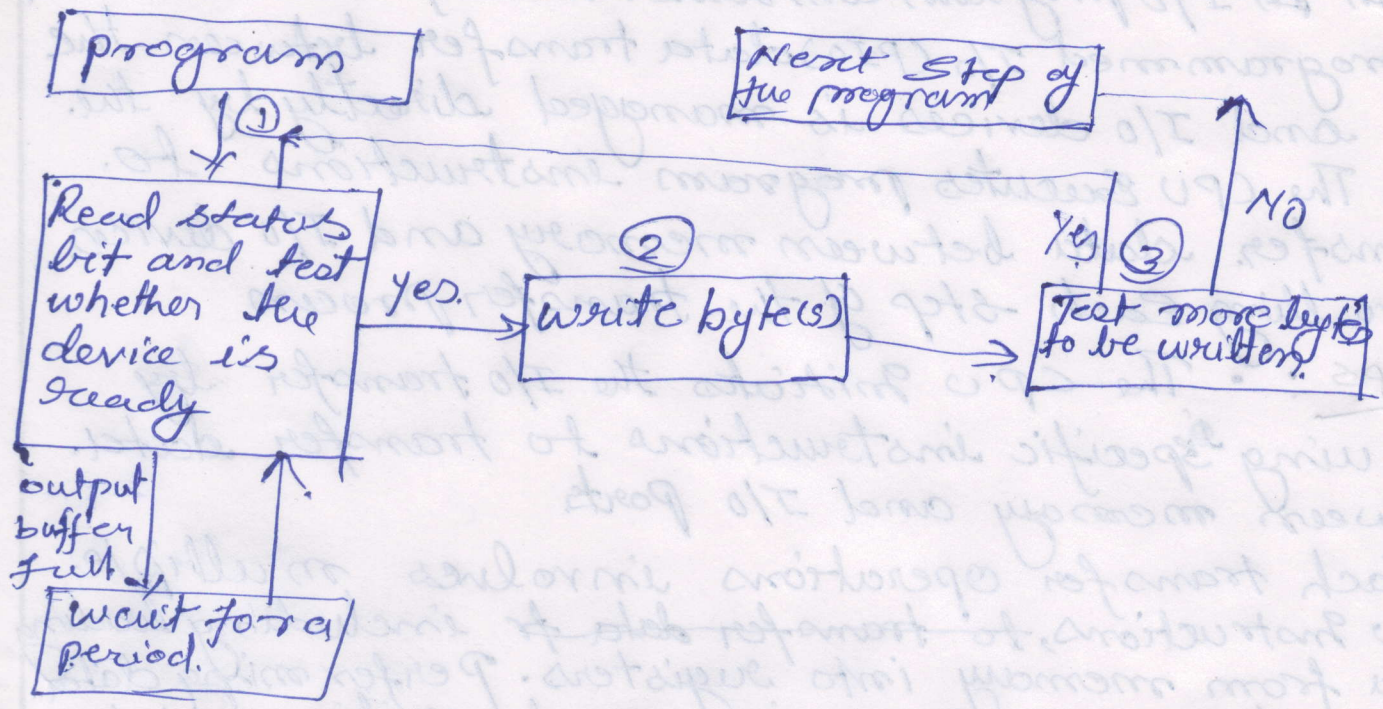


fig - Input read by the processor in programmed I/O mode



Output write by the processor in programmed I/O mode



Advantages: →

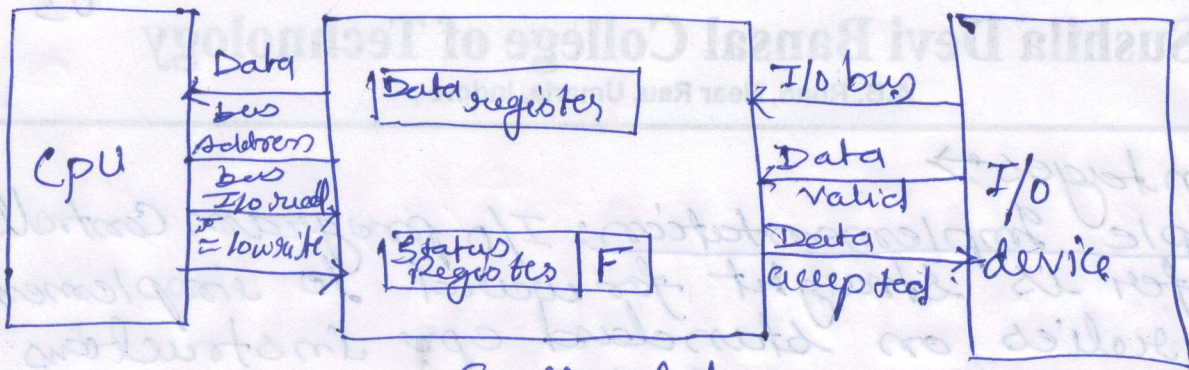
- Simple Implementation: I/O program controlled transfer is straight forward to implement as it relies on standard CPU instructions and programming techniques.
- Flexibility: CPU has full control over the data transfer process allowing for customization based on application specific requirements.

Disadvantages: →

- CPU Overhead: programmed I/O operations consume CPU cycles and processing resources potentially leading to inefficiency and reduced overall system performance.
- Slow Data Transfer: CPU's involvement in managing data transfer imposes overhead and introduces latency, resulting in slower data transfer rates compared to other methods.

DMA Transfer (Direct Memory Access) →

It is a method of data transfer between memory and I/O devices that bypasses the CPU and allows for direct communication between the memory and the I/O device. A DMA controller is used to coordinate and manage the data transfer process independently of the CPU.



F = flag bit

Fig Data transfer from I/O device to CPU

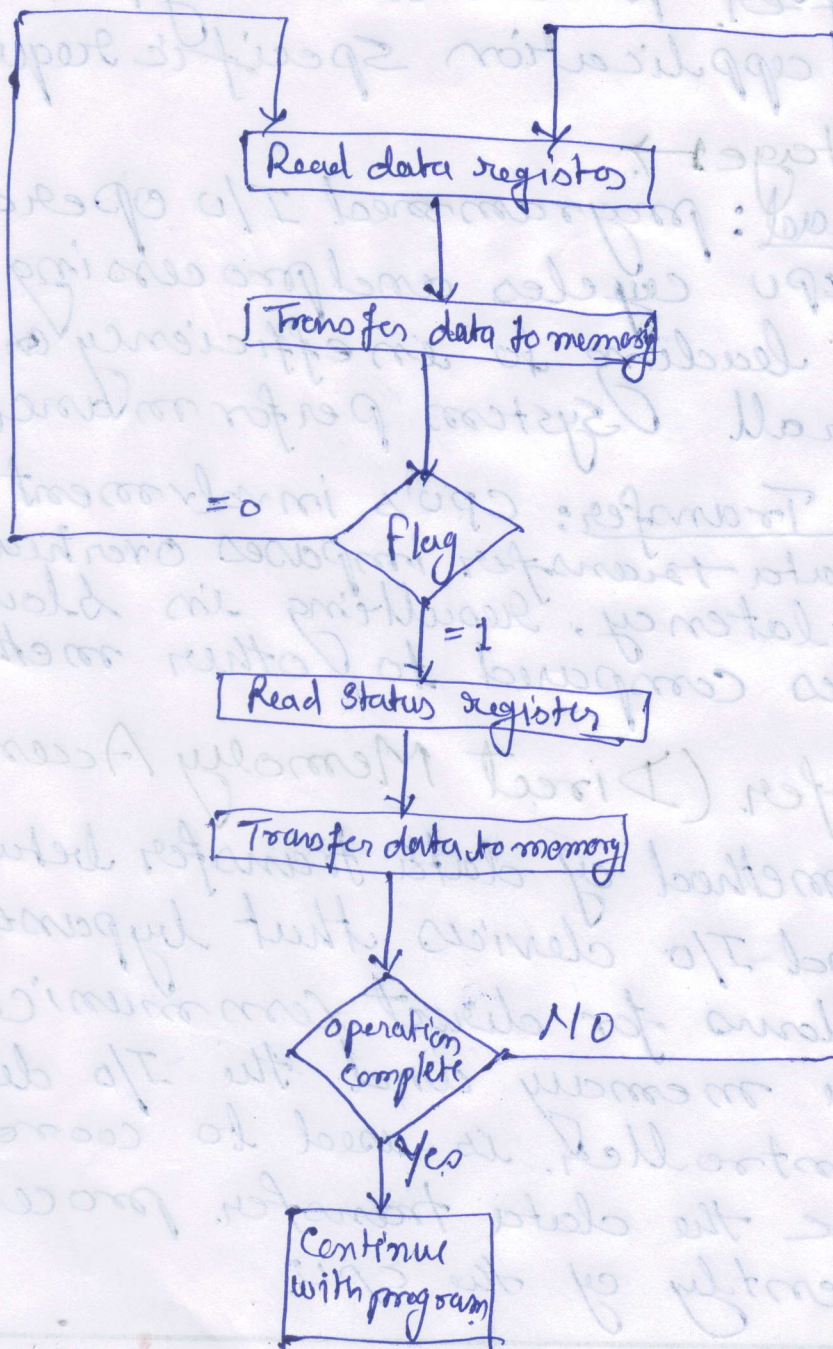
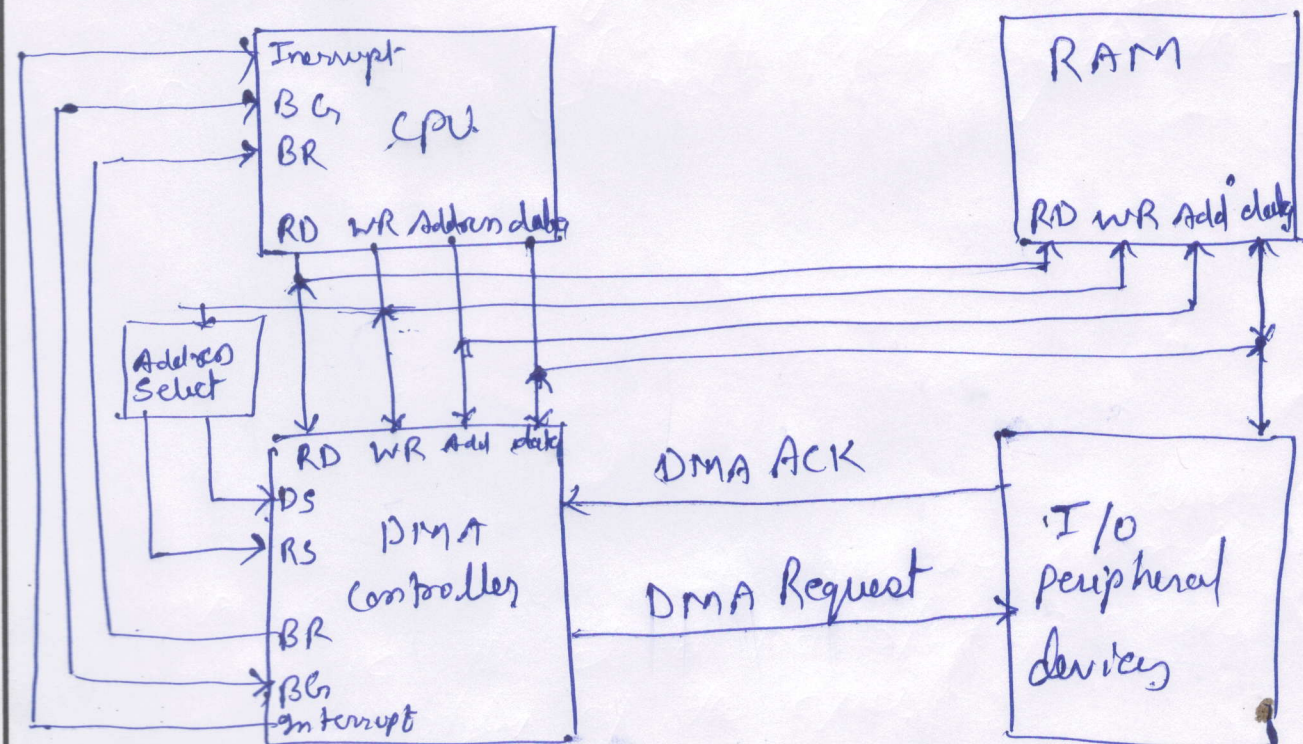


Fig. flow chart for CPU program to input data



Methods:-

- The DMA Controller is configured by the CPU with the source and destination addresses, transfer length, and transfer mode.
- Once configured the DMA Controller takes control of the system bus and initiates the data transfer between memory and I/O devices without CPU intervention.
- The DMA Controller manages data transfer operations, including memory access, data movement, and interrupt generation allowing the CPU to continue executing other tasks concurrently.



• DMA transfer in Computer Systems.



Advantages:-

- Increased throughput: It improves system performance and throughput by offloading data transfer tasks from ~~memory~~ the CPU, allowing the CPU to focus on executing application code.
- Reduced CPU Overhead: By Bypassing the CPU for data transfer operations, DMA transfer reduces CPU overhead and frees up CPU resources for other tasks.

Disadvantages:-

- Complexity: Implementing DMA transfer requires additional hardware components, such as DMA controllers, and entails more complex system design and configuration.
 - Limited Flexibility: DMA transfer may not be suitable for all types of data transfer operations or devices and configuration and management of DMA channels may require careful consideration.
- Flags in 8085 processor.
1. Sign flag (S), (2) Zero flag (Z), (3) Auxiliary Carry flag (AC), (4) Parity flag (P), (5) Carry flag (CY), (6) Overflow flag (O).



Q2. The size of the data count register of a DMA controller is 16 bits. The processor needs to transfer a file of 29,154 kilobytes from the disk to the main memory. The memory is byte addressable. The minimum number of times the DMA Controller needs to get control of the system bus from the processor to transfer the file from the disk to the main memory is?

Ans First we have to determine the total number of bytes to be transferred.

Given

- Size of the Data Count register of the DMA controller = 16 bits
- Size of the file to be transferred = 29,154 Kb

$$\text{Convert file size to bytes} = 29154 \text{ Kb} \times 1024 \text{ b/Kb} \\ = 29882496 \text{ bytes}$$

Calculate the minimum number of times DMA controller needs to transfer data

$$\begin{aligned} \text{No. of times} &= \frac{\text{Total bytes}}{\text{Max. data count register size}} \\ &= \frac{29882496}{(2^{16} \text{ bits})} \\ &= \frac{29882496}{65536 \text{ bits}} \\ &\approx 456 \end{aligned}$$

minimum no times DMA Controller needs

to get control of the system bus from the processor to transfer the file from disk to the main memory is approximately 456 times



Q3 Write and Elaborate notes on PCI, SCSI and USB bus standards. Also Explain these 03 interfaces in detail

Ans PCI - It is a computer bus standard that allows peripheral devices to be connected to a computer's mother board. It is commonly used for connecting expansion cards such as network adapters, sound cards and graphic cards.

Features

- High bandwidth
- Plug and play
- Bus Mastering
- Hot Swapping

Types

- PCI - offering data transfer rates upto 133 MB/s
- PCI/x - offering data transfer rates upto 1064 MB/s
- PCIe - offering data transfer rate upto 16 GB/s

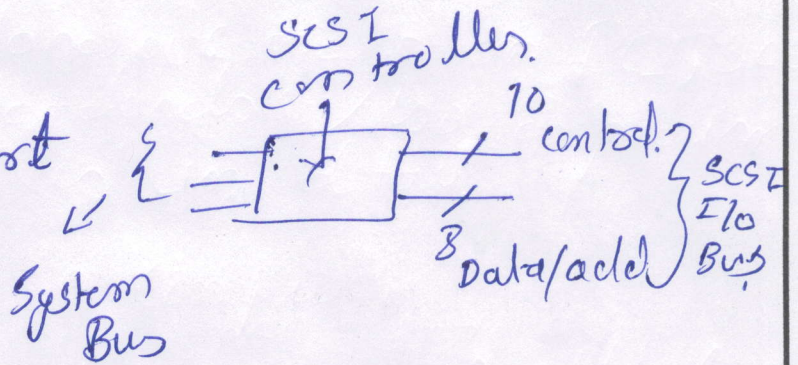
SCSI (Small Computer System Interface)

It is a set of standards for connecting and transferring data between computers and peripheral devices. Such as hard drives, tape drives, scanners. It provides high performance interface for storage devices.



Features

- 1) High Speed.
- 2) Multiple device Support
- 3) wide compatibility.
- 4) Hot Swapping.



Types

SCSI 1, SCSI 2, SCSI 3

USB (universal Serial Bus)

It is widely used standard for connecting peripheral devices to computers. like keyboard, mouse, printers and External Storage devices and it also provides a versatile and convenient interface for connecting wide range of devices.

Features:

- Plug & play
- Hot Swapping
- Power supply
- Scalability.

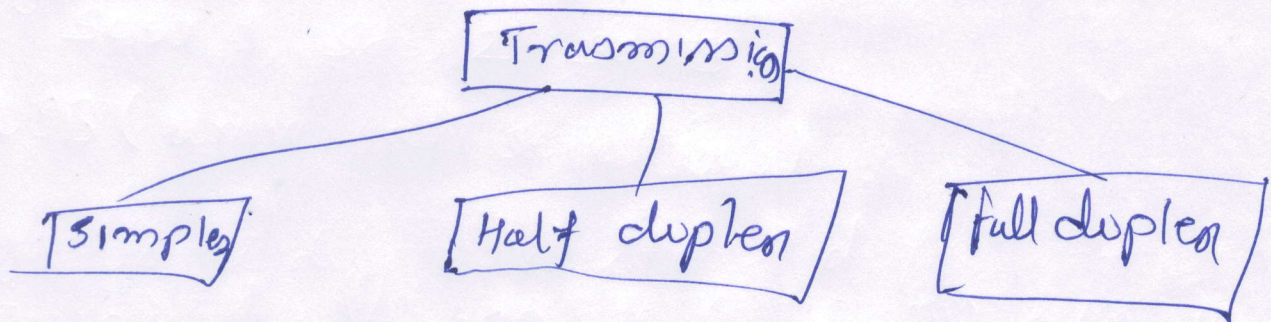
Types

- USB 1.1 offering data Speed transfer Speed upto 12 mb/s (USB 1.1) or 1.5 mbps (USB 1.0)
- USB 2.0 offering Speeds upto 480 mbps
- USB 3.0 - offering data Speeds 5 Gbps
- ~~PCI, SCSI and IS~~

Q4. With the help of suitable diagrams Explain Simplex, half duplex and full duplex transmission?

Ans ~~Transmission~~ ~~Between~~

There are three type of transmission mode

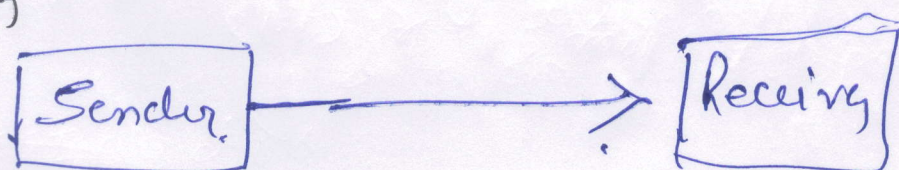


Simplex Transmission

It is a one way transmission or communication mode where data flows in only one direction from the Sender to the Receiver. The Receiver can not send data back to the Sender.

Ex - Television broadcasting where TV Station transmit signal to viewers without receiving any feedback.

Diagram



Half Duplex-

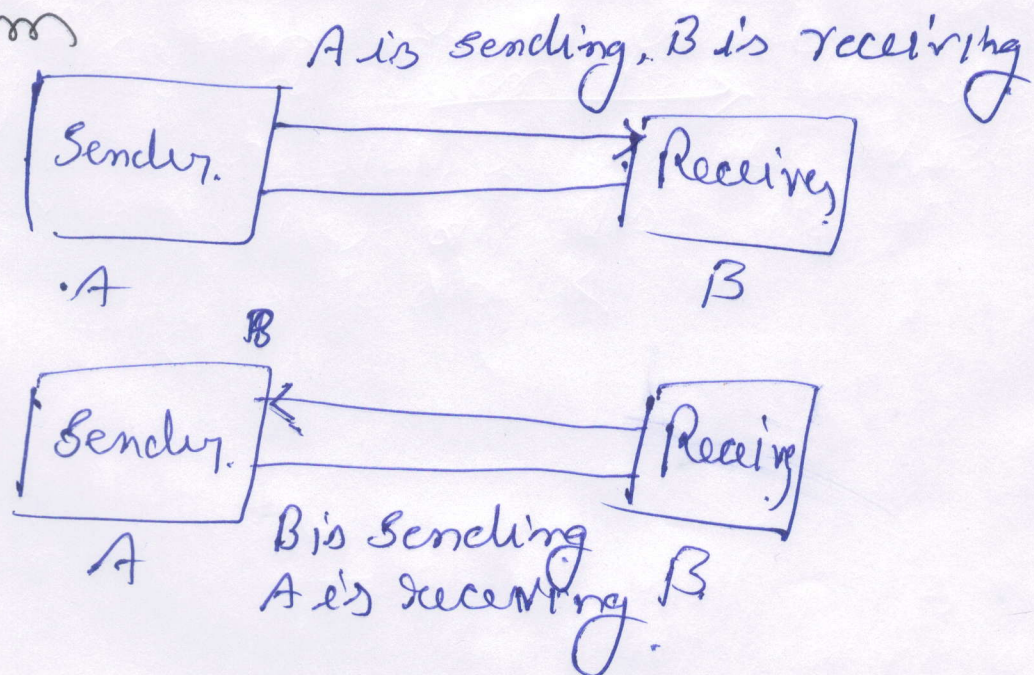
It is a two way communication mode where data ^{can} flow in both the directions. but not simultaneously.

The Communication channel alternates between sending and receiving data.

Example.

walkie-talkies. where users must push a button to transmit and release it to listen, allowing only one party to talk at a time.

Diagram





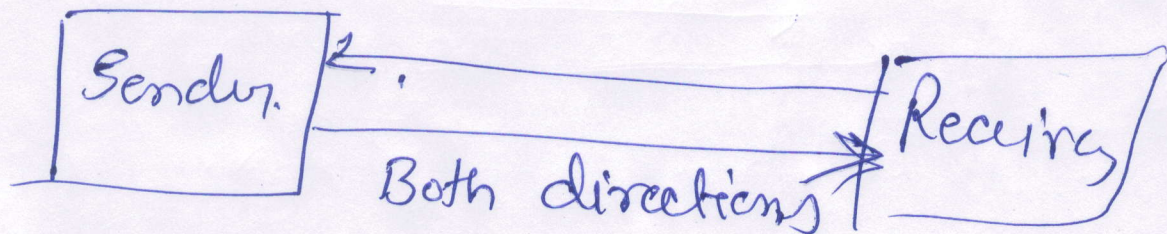
Full Duplex Transmission

It is a two way communication mode where data can flow simultaneously in both directions. This allows for faster and more efficient communication compared to half-duplex.

Example

Telephone. Conversations where both parties can speak and listen simultaneously without interrupting each other.

Diagram





Q5 Elaborate the following with Examples?

- (1) Serial and parallel data transfer
- (2) Synchronous and Asynchronous modes of data transfer.
- (3) Non-Vectored and Vectored interrupt
- (4) Maskable and Non-Maskable interrupt.
- (5) Hardware and Software Interrupt.

(1) Serial and parallel data transfer.

Serial data transfer.

It is a method of transmitting data one bit at a time over a single communication channel. In it data is sent sequentially, one after bit by bit, along a single wire or communication path.

Method

- Data is Serialized, meaning it is converted into a stream of bits.
- Each bit is transmitted sequentially, one after the other, over the communication path.
- Clock Synchronization is often used to ensure that the sender and receiver are synchronized in timing.

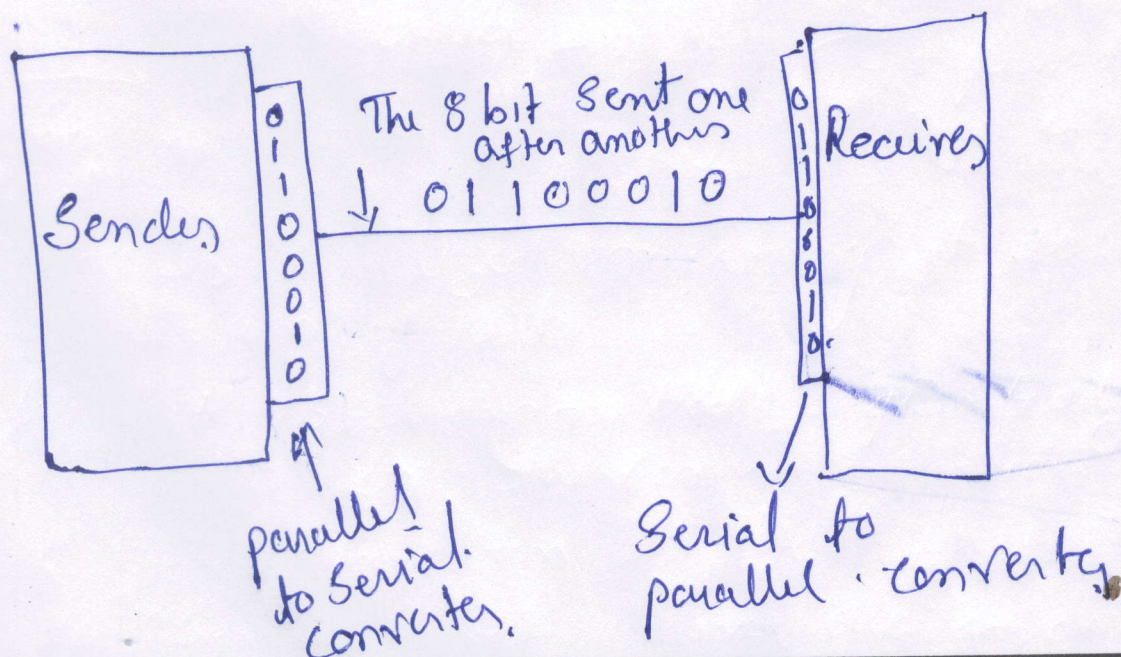
Advantage

- (1) Simplified wiring
- (2) Longer distance communication.
- (3) Less Expensive

Example.

- RS 232 Serial Communication for connecting devices like modems, printers, and serial ports on computers.
- USB (Universal Serial Bus) for connecting peripherals such as keyboards mouse, and External Storage devices to computer.
- Serial Communication between microcontroller and sensors in Embedded Systems.

Diagram.





parallel data transfer -

It is a method of transmitting multiple bits simultaneously over multiple communication channels. In it data is sent in parallel meaning multiple bits are transmitted simultaneously over separate wires or communication paths.

Method:

- Data is divided into multiple parallel channels, with each channel carrying one bit of the data word.
- All bits of the data word are transmitted simultaneously each on its own wire or communication path.
- Requires synchronization to ensure that all bits arrived at the receiver at the same time.

Advantages

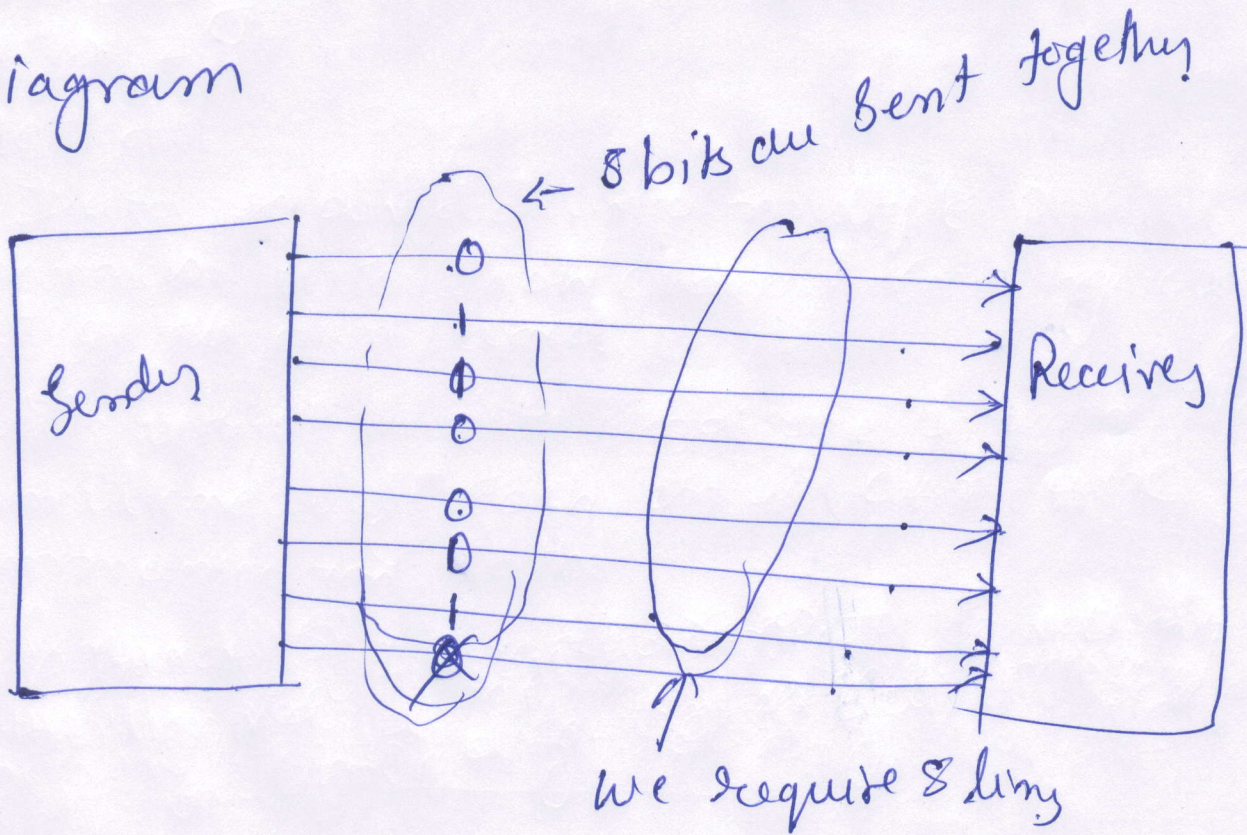
- (1) Higher data transfer rates.
- (2) Simultaneous transmission to reduce overall transmission time.
- (3) Parallel processing of data at the receiver end enabling faster data processing.

Example

parallel ports for connecting printers, scanners and other peripheral devices

- Internal data buses in computers such as the system bus and memory bus for transferring data between components
- Inter chip communication in IC's and microprocessors.

Diagram



Parallel transmission



11) Synchronous and Asynchronous modes of data transfer

Synchronous data transfer mode

It is a method of data communication where data is transmitted in synchronized timing with a common clock signal shared between sender and receiver.

Method:

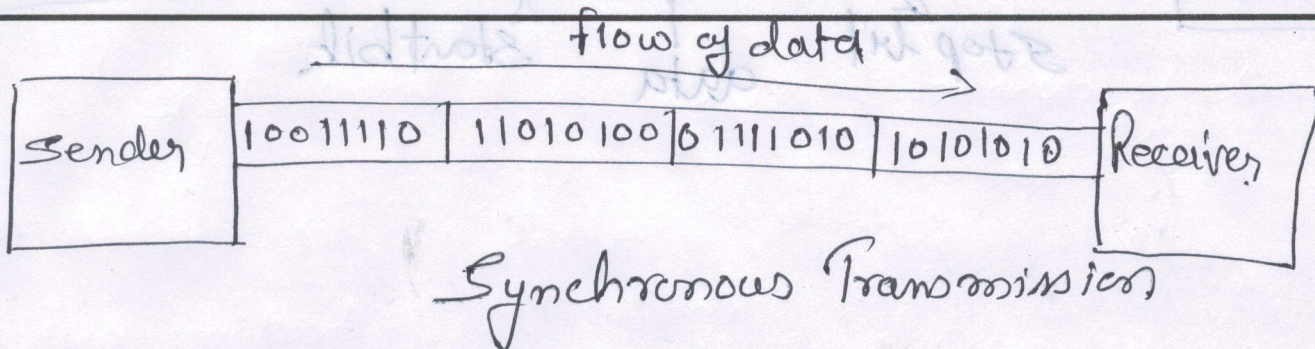
- Both the sender and receiver are synchronized to a common clock signal.
- Data is transmitted in fixed size blocks or frames with each frame synchronized to the clock signal.
- The clock signal ensures that data is sampled at precise intervals allowing for accurate transmissions and receptions.

Advantages

- Reliable
- Efficient
- Scalable

Examples

- Synchronous Serial Interface (SSI) for communication between microcontrollers and peripheral devices.
- Synchronous Optical Networking (SONET)
- Synchronous transmission protocols (SCTP)



2) Asynchronous Data transfer

It is a method of data transfer without a synchronized clock signal. Instead start and stop bits are used to indicate the beginning and end of each data byte.

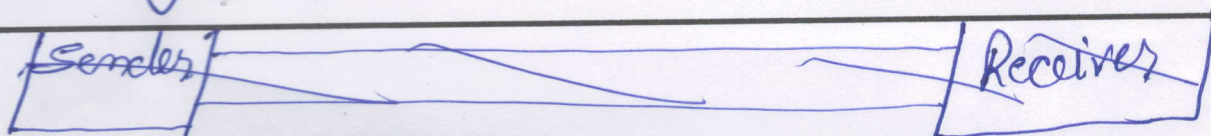
Method:

- Each data byte is framed by start and stop bits, which define the boundaries of the data byte.
- Data is transmitted asynchronously, meaning there is no fixed timing of data relationship between sender and receiver.
- Both the sender and receiver use their own clocks to determine the timing of data transmission and reception.

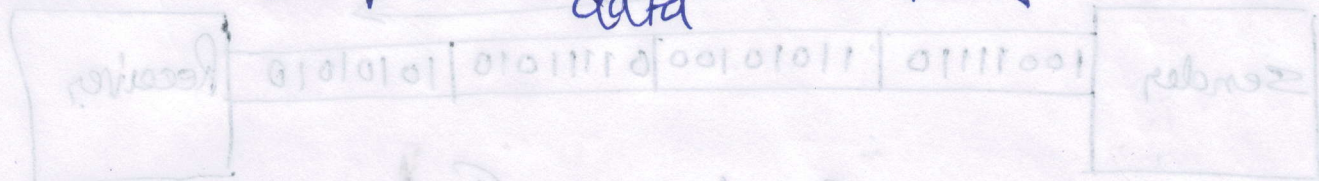
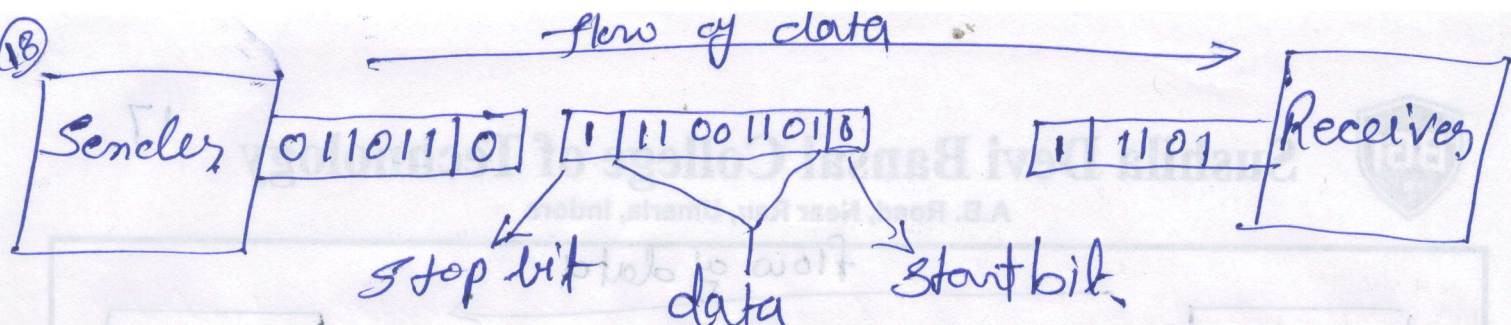
Advantages

1) Simple, 2) flexible, 3) Cost-effective

Example:- Asynchronous Transfer mode (ATM) for high speed network data transmission



18



Synchronous transmission

Asynchronous data transfer

It is a method of data transfer without a synchronized clock signal. Instead, start and stop bits are used to indicate the beginning and end of each data byte.

Method:

- Each data byte is framed by start and stop bits, which define the boundaries of the data byte.

- Data is transmitted asynchronously, meaning there is no fixed timing of data relationship between sender and receiver.
- Both the sender and receiver use their own clocks to determine the timing of data transmission and reception.

Advantages

- Simple
- flexible
- Cost-effective

Example: - Asynchronous transfer mode (ATM) for high speed network data transmission



3) Vectored and non-vector interrupt.

Vector interrupts are those interrupt which have fixed vector address (starting address of sub routine) and after executing these program control is transfer to that address.

Vector interrupt handling

Each interrupt source is assigned a unique vector. interrupt vector which is a memory address pointing to the corresponding interrupt service routine (ISR). When an interrupt occurs, the processor can directly jump to the appropriate ISR based on the interrupt vector, without the need for sequential checks.

Steps

- Each interrupt source is assigned a unique interrupt vector, typically stored in a dedicated interrupt vector table in memory.
- When an interrupt occurs, the interrupt controller provides the processor with the interrupt vector corresponding to the interrupt source.
- The processor uses the interrupt vector to directly jump to the corresponding ISR, bypassing the need for sequential checks of interrupt sources.



Non Vector Interrupt-

In this vector address is not predefined. The interrupting device gives the address of Sub routine for these interrupts.

INTR is the only non-vectorised interrupt in 8085 processor.

Non-Vectorised handling

In this there is no specific mechanism for identifying the source of interrupt. When an interrupt occurs, the processor must sequentially check each interrupt source to determine which device or service request generated the interrupt.

Steps

- When an interrupt occurs, the processor suspends the current program execution and enters the interrupt service routine (ISR).
- The ISR sequentially checks each interrupt source, typically through a polling mechanism, to determine which device or service request generated the interrupt.
- Once the interrupt source is identified, the ISR performs the necessary actions to handle the interrupt, such as servicing the device or responding to the service request.



4) Maskable and Non-Maskable Interrupts

Maskable Interrupts

Maskable Interrupts are those Interrupts that can be temporarily disabled (masked) by the processor during certain critical sections of code execution. The processor has control over whether to enable or disable maskable interrupts.

Steps

- When a maskable interrupt occurs, the processor checks the current interrupt mask status.
- If maskable interrupts are enabled (unmasked) the processor suspends the ~~current~~ current program execution and enters the interrupt service request (ISR) to handle the interrupt.
- If maskable interrupts are disabled (masked), the processor ignores the interrupt request and continues executing the current program without interruption.
- Maskable interrupts can be enabled or disabled using specific instructions or by setting/clearing flags in the interrupt control register.



Non-maskable Interrupts (NMI)

Non-maskable interrupts are interrupts that can not be disabled (masked) by the processor. NMIs are designed to be prioritized over all other interrupt sources and are typically used for critical system events that requires immediate attention.

Steps

- When an NMI occurs, the processor quickly suspends the current program execution and enters the NMI handler routine, regardless of the current interrupt mask status.
- NMIs cannot be disabled or ignored by the processor and take precedence over all other interrupt sources.



5} Hardware and Software Interrupts.

Hardware Interrupts

hardware interrupts are signals generated by external hardware devices to interrupt the normal flow of program execution in the processor. These interrupts are triggered by hardware events such as input/output (I/O) operations, timer expiration and hardware faults.

Handling Method

- When a hardware interrupt occurs, the processor suspends the current program execution and jumps to a predefined interrupt service routine (ISR) to handle the interrupt.
- The ISR typically performs actions such as servicing the hardware device, acknowledging the interrupt and restoring the state of the interrupted program.
- Hardware interrupts are prioritized based on their importance and urgency, with higher priority interrupts taking precedence over lower priority ones.



Examples -

- I/O interrupts → Generated by peripheral devices
- Timer interrupts → Generated by hardware timers to signal the expiration of predetermined time interval
- Hardware fault interrupts

- Software Interrupts

also known as traps or Exceptions are signals generated by the processors or software to request a change in the normal execution flow. Unlike hardware interrupts, software interrupts are triggered by software events or instructions rather than external hardware events.

Handling Method

- When software interrupt occurs the processor transitions to kernel mode and executes a predefined IR, ISR or Exception handler to handle the interrupt
- Software interrupts are commonly used for system calls, error handling and other software initiated actions that require privileged access to system resources.

Example. System call, Exception handling, Interrupt instructions